UNIVERSITATEA TEHNICĂ DIN CLUJ-NAPOCA

Forum Application

Proiectare Software

Autori:

Leordean Alina, Sebeș Ruxandra
și Munteanu George

Grupa: 30238

Facultatea de Automatica
si Calculatoare

24 Martie 2025

Cuprins

1 Assignment 1 . . . . . **2**
1.1 Maparea Entităților 2
1.2 Operațiuni CRUD 3
1.2.1 CRUD User 3
1.2.2 CRUD Question 5
1.2.3 CRUD Answer 6
1.3 Managementul Bazei de Date 7
1.4 Testare 7
1.5 Diagrame 9

2 Assignment 2 . . . . . **11**
2.1 Implementare Interfață Utilizator 11
2.2 Utilizare JSON 11
2.3 Modularizare și Servicii 11
2.4 Calitate Cod și Testare 11

3 Assignment 3 . . . . . **11**
3.1 Dezvoltare Funcționalități Avansate 11
3.2 Arhitectură și Microservicii 11
3.3 Testare End-to-End (E2E) 11

1 Assignment 1

1.1 Maparea Entităților

Această secțiune documentează entitățile din proiect, definind structura și relațiile acestora în baza de date. Maparea entităților este procesul prin care clasele dintr-o aplicație sunt asociate cu tabelele din baza de date, permițând manipularea datelor într-un mod orientat pe obiecte.

Prin mapare, se asigură o interacțiune eficientă și organizată între aplicație și baza de date, reducând complexitatea codului și facilitând gestionarea datelor.

User

Entitatea `User` reprezintă utilizatorii platformei. Aceasta conține informații precum email, username, rol și scor.

- **ID:** Cheie primară, generată automat.
- **Email și username:** Unice și obligatorii.
- **Rol:** Definește permisiunile utilizatorului.
- **Scor:** Valoare numerică implicită 0.
- **Este interzis (is_banned):** Flag boolean pentru interzicerea accesului.

Question

Entitatea `Question` reprezintă întrebările postate de utilizatori.

- **ID:** Cheie primară, generată automat.
- **Autor:** Relație `@ManyToOne` cu `User`.
- **Titlu și conținut:** Informațiile întrebării.
- **Dată creare:** Stocată automat la inserare.
- **Status:** Starea întrebării.

Answer

Entitatea `Answer` reprezintă răspunsurile oferite la întrebări.

- **ID:** Cheie primară.
- **Întrebare:** Relație `@ManyToOne` cu `Question`.
- **Autor:** Relație `@ManyToOne` cu `User`.
- **Conținut și imagine:** Textul și eventual o imagine atașată.
- **Dată creare:** Setată automat la inserare.

Tag

Entitatea `Tag` reprezintă etichetele folosite pentru clasificarea întrebărilor.

- **ID:** Cheie primară.
- **Nume:** Unic și obligatoriu.

QuestionTag

Entitate intermediară pentru relația `@ManyToMany` între `Question` și `Tag`.

- **Cheie compusă:** `question_id` și `tag_id`.

Vote

Entitatea `Vote` reprezintă voturile utilizatorilor asupra întrebărilor și răspunsurilor.

- **ID:** Cheie primară.
- **Utilizator:** Relație `@ManyToOne` cu `User`.
- **Întrebare sau răspuns:** `@ManyToOne` cu `Question` sau `Answer`.
- **Tip vot:** Valoare numerică pentru vot pozitiv/negativ.

Ban

Entitatea `Ban` gestionează interzicerea utilizatorilor.

- **ID:** Cheie primară.
- **Utilizator:** Relație `@OneToOne` cu `User`.
- **Moderator:** Relație `@ManyToOne` cu `User`.
- **Motiv și dată:** Justificare și momentul interdicției.

Adnotările JPA utilizate

- `@Entity`: Marchează o clasă ca fiind o entitate JPA, mapată la o tabelă din baza de date.
- `@Table(name = "nume_tabel")`: Specifică numele tabelului asociat entității.
- `@Id`: Definește câmpul care servește drept cheie primară.
- `@GeneratedValue(strategy = GenerationType.IDENTITY)`: Indică faptul că ID-ul este generat automat de baza de date.
- `@Column(name = "nume_coloană")`: Specifică detalii despre coloana mapată.
- `@ManyToOne`: Definește o relație de tip „mulți-la-unu” între entități.
- `@OneToOne`: Definește o relație de tip „unu-la-unu” între entități.
- `@ManyToMany`: Definește o relație de tip „mulți-la-mulți” între entități.
- `@JoinColumn(name = "nume_coloană")`: Specifică coloana străină utilizată pentru mapare.
- `@Temporal(TemporalType.TIMESTAMP)`: Indică faptul că un câmp `Date` este stocat ca timestamp în baza de date.

Maparea entităților este realizată utilizând JPA, definind relațiile dintre entități prin anotări adecvate. Acest model de date facilitează gestionarea utilizatorilor, întrebărilor, răspunsurilor, voturilor și interdicțiilor într-un sistem bine structurat.

1.2 Operațiuni CRUD

1.2.1 CRUD User

Această documentație descrie operațiile CRUD pentru entitatea `User` într-o aplicație Spring Boot, evidențiind structura și responsabilitățile fiecărei clase.

Structura permite gestionarea utilizatorilor într-un mod clar și modular, separând responsabilitățile între diferitele componente ale aplicației. Utilizarea `CrudRepository` simplifică interacțiunea cu baza de date, iar serviciul intermediază logica de business, asigurând un design flexibil și ușor de extins.

Arhitectura și Structura

Implementarea se bazează pe arhitectura standard Model-View-Controller (MVC):

- **Entity**: Reprezintă modelul de date al utilizatorului.
- **Repository**: Interfață pentru interacțiunea cu baza de date.
- **Service**: Conține logica de business și intermediază între repository și controller.
- **Controller**: Expune endpoint-uri REST pentru operațiile CRUD.

Implementare

UserRepository

Interfața `UserRepository` extinde `CrudRepository` pentru a permite operațiile de bază asupra bazei de date.

```
@Repository
public interface UserRepository extends CrudRepository<User, Long> {
}
```

UserService

Clasa `UserService` gestionează logica aplicației și oferă metode pentru manipularea entităților `User`.

```
@Service
public class UserService {
    @Autowired
    private UserRepository userRepository;

    public User findUserById(Long id) {
        Optional<User> userOptional = userRepository.findById(id);
        return userOptional.orElseThrow(() ->
              new RuntimeException("User not found with id: " + id));
    }

    public List<User> getAllUsers() {
        return (List<User>) userRepository.findAll();
    }

    public User insertUser(User user) {
        return userRepository.save(user);
    }

    public User updateUser(User user) {
        return userRepository.save(user);
    }

    public String deleteUser(Long id) {
        try {
```

```
            userRepository.deleteById(id);
            return "Deletion Successfully";
        } catch (Exception e) {
            return "Failed to delete user with id " + id;
        }
    }
}
```

UserController

Clasa `UserController` expune operațiile CRUD prin endpoint-uri REST.

```
@RestController
@RequestMapping("/users")
public class UserController {
    @Autowired
    private UserService userService;

    @GetMapping("/getAll")
    public List<User> getAllUsers() {
        return userService.getAllUsers();
    }

    @PostMapping("/insert")
    public User insertUser(@RequestBody User user) {
        return userService.insertUser(user);
    }

    @PutMapping("/update")
    public User updateUser(@RequestBody User user) {
        return userService.updateUser(user);
    }

    @DeleteMapping("/deleteById")
    public String deleteUser(@RequestParam Long id) {
        return userService.deleteUser(id);
    }

    @GetMapping("/test")
    public String test() {
        return "User controller is working!";
    }
}
```

1.2.2 CRUD Question

Arhitectură și Structură

Gestionarea întrebărilor este realizată prin trei componente principale:

- **QuestionService** – gestionează logica de afaceri și interacționează cu repository-ul.
- **QuestionRepository** – interfață care permite accesul la datele persistente.
- **QuestionController** – definește API-ul REST pentru operațiile de manipulare a întrebărilor.

Operații CRUD

Obținerea tuturor întrebărilor

Metoda `getAllQuestions()` returnează lista întrebărilor stocate în baza de date.

Endpoint: `GET /questions/getAll`.

Inserarea unei întrebări

Metoda `insertQuestion(Question question, Long autorId)` adaugă o nouă întrebare.

Endpoint: `POST /questions/insert`, primește un obiect `Question` în `RequestBody`, id-ul autorului întrebării în RequestParam.

Actualizarea unei întrebări

Metoda `updateQuestion(Question question, Long questionId, Long authorId)` modifică datele unei întrebări existente.

Endpoint: `PUT /questions/update`, primește un obiect `Question` actualizat în RequestBody id-ul autorului și id-ul întrebării în RequestParam.

Ștergerea unei întrebări

Metoda `deleteQuestion(Long id)` elimină o întrebare din sistem.

Endpoint: `DELETE /questions/deleteById?id=ID`.

1.2.3 CRUD Answer

Modulul de gestionare a răspunsurilor

Modulul de gestionare a răspunsurilor (`Answer`) permite utilizatorilor să răspundă la întrebările existente, să își gestioneze propriile răspunsuri și să interacționeze cu alte răspunsuri prin voturi.

În metoda *insertAnswer*, corpul (@RequestBody) conține obiectul Answer, care include conținutul răspunsului, iar parametrii (@RequestParam) authorId și questionId sunt folosiți pentru a identifica utilizatorul care postează răspunsul și întrebarea asociată.

În metoda *updateAnswer*, corpul (@RequestBody) conține obiectul Answer actualizat (inclusiv id, content), iar parametrii (@RequestParam) authorId și questionId sunt utilizați pentru a valida apartenența răspunsului la un utilizator și o întrebare existente.

Structură

CRUD-ul pentru `Answer` este implementat prin:

- **AnswerService** – se ocupă de logica de afaceri.
- **AnswerRepository** – interfață pentru interacțiunea cu baza de date.

- **AnswerController** – expune endpointurile REST pentru manipularea răspunsurilor.

Relaționare între entități

- Fiecare răspuns este asociat unei **întrebări**.
- Răspunsurile sunt legate de un **autor**.
- Opțional, un răspuns poate avea o **imagine**.
- Răspunsurile pot fi **votate**, iar numărul total de voturi influențează ordinea lor.

Alte Funcționalități

- **Votare**: Răspunsurile pot primi upvote-uri și downvote-uri.
- **Sortare**: Răspunsurile sunt afișate în funcție de numărul de voturi.
- **Impact asupra întrebărilor**: Prima interacțiune cu un răspuns schimbă starea întrebării în „În curs de rezolvare”.
- **Autorizare**: Răspunsurile pot fi adăugate doar de utilizatorii autentificați.

1.3 Managementul Bazei de Date

Pentru asigurarea funcționalității și integrității bazei de date **Assignment**, au fost efectuate mai multe teste și verificări folosind *Postman* și interogări directe în *SQL*.

Testarea API-ului a inclus verificarea corectitudinii operațiilor **CRUD** pentru întrebări și răspunsuri. Fiecare rută a fost testată prin trimiterea de cereri cu diferite seturi de date pentru a valida răspunsurile serverului, gestionarea erorilor și respectarea constrângerilor de integritate. În special, au fost verificate următoarele aspecte:

- **Crearea de înregistrări** – Testarea endpointurilor `POST` pentru adăugarea de întrebări și răspunsuri, asigurând că datele sunt salvate corect în baza de date.
- **Modificarea datelor** – Verificarea actualizării înregistrărilor prin metodele `PUT`, confirmând că datele sunt modificate corespunzător.
- **Ștergerea înregistrărilor** – Testarea endpointurilor `DELETE` pentru eliminarea întrebărilor și răspunsurilor și verificarea efectelor în baza de date.
- **Gestionarea erorilor** – Simularea scenariilor cu date incomplete, ID-uri inexistente sau cereri incorecte pentru a verifica validarea și mesajele de eroare corespunzătoare.
- **Respectarea relațiilor** – S-a verificat dacă eliminarea unei întrebări duce la ștergerea automată a răspunsurilor asociate (dacă este aplicată o ștergere în cascadă) sau dacă este gestionată corect prin constrângeri de integritate.
  În plus, s-a constatat că, pentru a putea șterge un utilizator, este necesar ca mai întâi să fie șterse toate întrebările acestuia, iar înainte de acestea, trebuie eliminate răspunsurile asociate. Acest proces asigură menținerea integrității referențiale și prevenirea erorilor la ștergere.

1.4 Testare

Testare Unitare cu JUnit și Mockito

Structura Clasei de Test

- **Pachet**: `package serviceTest;`

- **Importuri**: Clasele necesare din JUnit 5 (`org.junit.jupiter.api`), Mockito (`org.mockito`) și alte pachete relevante.
- **Clasă**: `UserServiceTest`

Anotări și Atribute

- `@ExtendWith(MockitoExtension.class)`: Activează suportul pentru Mockito în JUnit 5, permițând utilizarea anotărilor `@Mock` și `@InjectMocks`.
- `@Mock`: Simulează o instanță a clasei `UserRepository`.
- `@InjectMocks`: Creează o instanță a clasei `UserService` și injectează mock-urile definite cu `@Mock`.

Metode de Test

testInsertUser()

Scop: Testează funcționalitatea de inserare a unui utilizator.

1. Se creează un obiect `User` cu valori specifice.
2. Se configurează mock-ul `userRepository` să returneze același obiect `User` când este apelată metoda `save`.
3. Se apelează metoda `insertUser` din `userService`.
4. Se verifică dacă rezultatul nu este `null` și dacă username-ul este corect.
5. Se verifică dacă metoda `save` a fost apelată o singură dată.

testGetAllUsers()

Scop: Testează funcționalitatea de obținere a tuturor utilizatorilor.

1. Se creează două obiecte `User` cu valori diferite.
2. Se configurează mock-ul `userRepository` să returneze o listă cu cele două obiecte când este apelată metoda `findAll`.
3. Se apelează metoda `getAllUsers` din `userService`.
4. Se verifică dacă dimensiunea listei returnate este corectă.
5. Se verifică dacă metoda `findAll` a fost apelată o singură dată.

testFindUserByIdSuccess()

Scop: Testează funcționalitatea de căutare a unui utilizator după ID cu succes.

1. Se creează un obiect `User` cu un ID specific.
2. Se configurează mock-ul `userRepository` să returneze acest obiect când este apelată metoda `findById` cu ID-ul respectiv.
3. Se apelează metoda `findUserById` din `userService`.
4. Se verifică dacă rezultatul nu este `null` și dacă username-ul este corect.

testFindUserByIdNotFound()

Scop: Testează funcționalitatea de căutare a unui utilizator după ID când acesta nu există.

1. Se configurează mock-ul `userRepository` să returneze `Optional.empty()` când este apelată metoda `findById` cu un ID inexistent.
2. Se apelează metoda `findUserById` din `userService`.

3. Se verifică dacă este aruncată o excepție `RuntimeException`.

testDeleteUserSuccess()

Scop: Testează funcționalitatea de ștergere a unui utilizator cu succes.

1. Se configurează mock-ul `userRepository` să nu facă nimic când este apelată metoda `deleteById` cu un ID specific.
2. Se apelează metoda `deleteUser` din `userService`.
3. Se verifică dacă mesajul returnat este `"Deletion Successfully"`.
4. Se verifică dacă metoda `deleteById` a fost apelată o singură dată.

testDeleteUserFailure()

Scop: Testează funcționalitatea de ștergere a unui utilizator când apare o eroare.

1. Se configurează mock-ul `userRepository` să arunce o excepție `RuntimeException` când este apelată metoda `deleteById` cu un ID specific.
2. Se apelează metoda `deleteUser` din `userService`.
3. Se verifică dacă mesajul returnat este `"Failed to delete user with id 99"`.

1.5 Diagrame

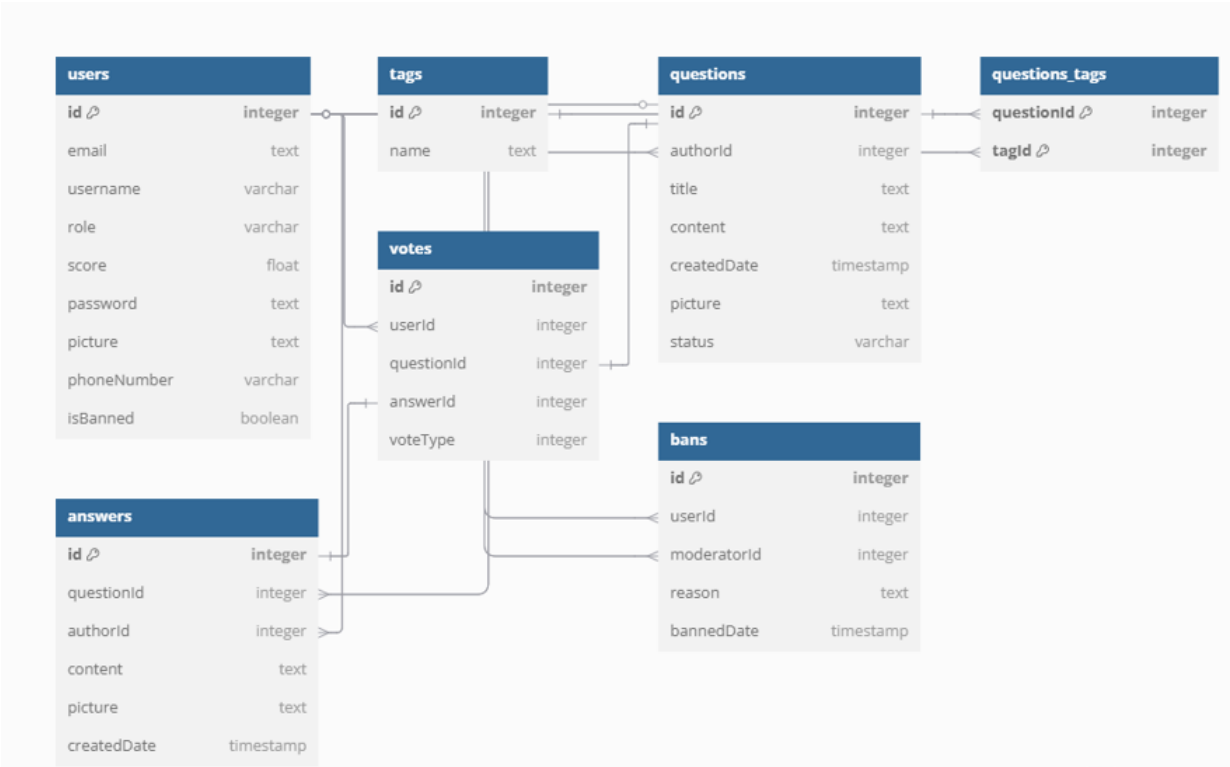

Figura 1: Diagrama bazei de date

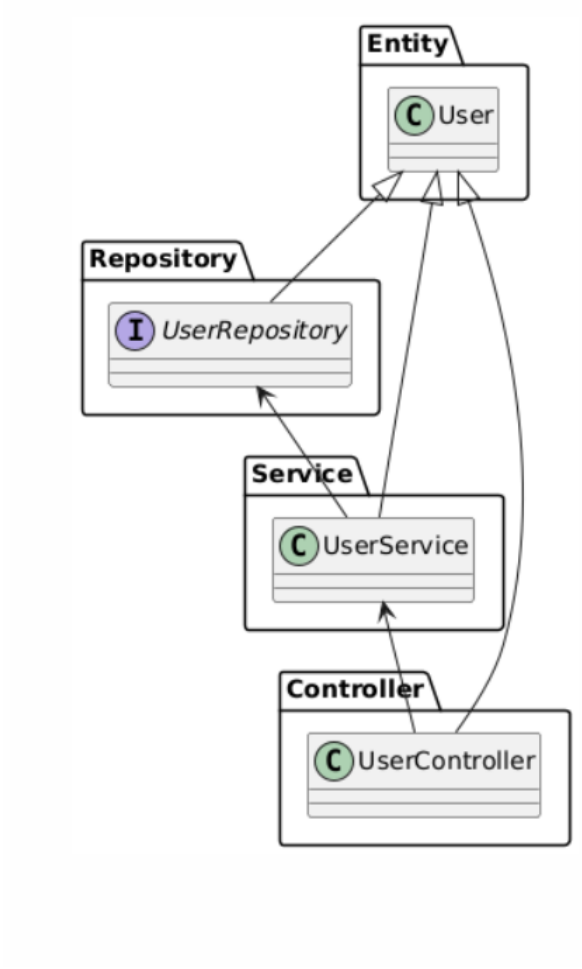

Figura 2: Diagrama de pachete

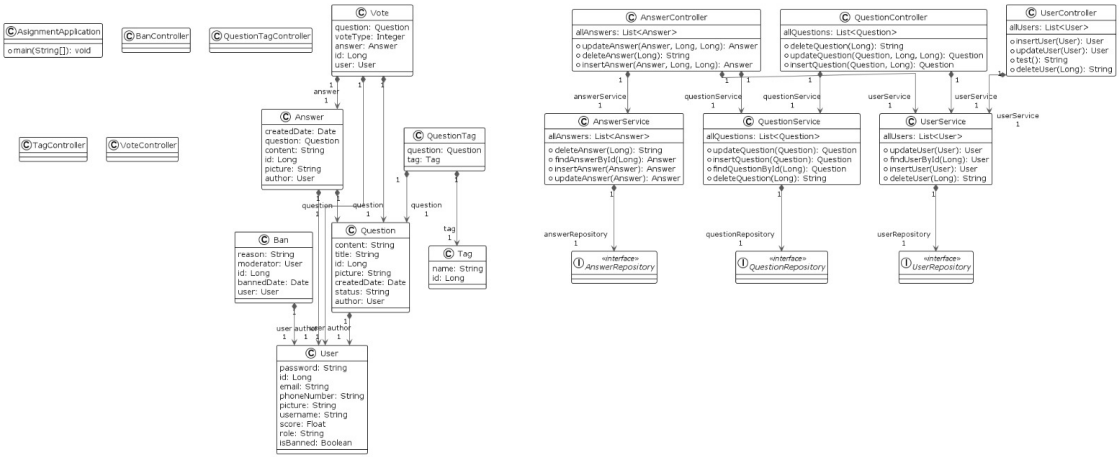

Figura 3: Diagrama de clase

2 Assignment 2

2.1 Implementare Interfață Utilizator

2.2 Utilizare JSON

2.3 Modularizare și Servicii

2.4 Calitate Cod și Testare

3 Assignment 3

3.1 Dezvoltare Funcționalități Avansate

3.2 Arhitectură și Microservicii

3.3 Testare End-to-End (E2E)